

3: Dateiverwaltung

TUTORIUM 24 - BuS 2024

Frage 1:

In manchen Betriebssystemen gibt es den Systemaufruf **rename**. Gibt es beim Umbenennen einer Datei einen Unterschied (auch in Hinblick auf die Performance) zwischen dem **rename**-Aufruf und dem **Kopieren** dieser Datei in eine **neue Datei** mit **neuem Namen**, mit anschließendem **Löschen der Ursprungsdatei**?

Frage 1: Unterschied zwischen `rename` und Kopieren in neue Datei mit anschließendem Löschen fürs Umbenennen einer Datei

- Dateisysteme wie FAT oder EXT aus der Vorlesung kopieren
beim Kopieren einer Datei diese Datei blockweise und erzeugen eine neue Inode für diese Datei
- `rename` ändert nur den Namen der Datei innerhalb des Verzeichnisses. Es muss nur der Verzeichnisblock den neuen Namen aufnehmen
- **Fazit:** `rename` ist vor allem für große Dateien viel effizienter

Frage 2:

Angenommen wir befinden uns gerade in `/home/root/`.

Was wäre der **absolute** Dateipfad für die Datei dessen **relativer** Dateipfad `../redha/BuS/sched.txt` ist?

Erinnerung

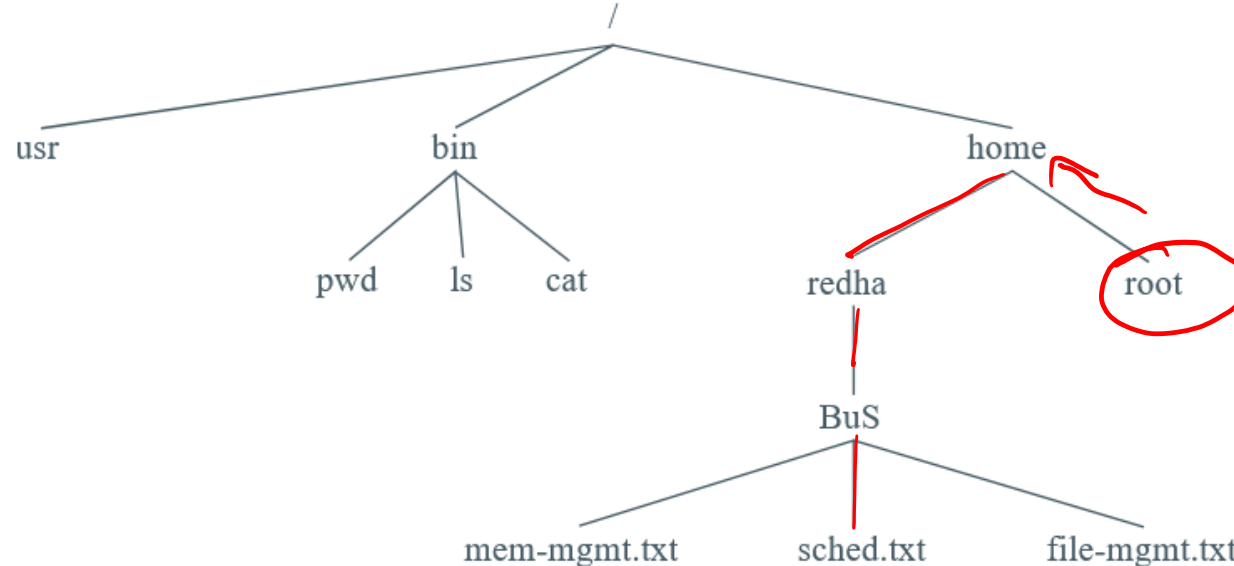
absoluter Dateipfad: kompletter Dateipfad von der Wurzel aus gelesen

relativer Dateipfad: Dateipfad vom **cwd** (*Current Working Directory*, also da wo wir uns gerade befinden) aus gelesen

Frage 2: Absoluter Dateipfad vom relativen Dateipfad

`../redha/BuS/sched.txt` mit `cwd = /home/root/`

- `..` geht ins Parent directory, also ein Verzeichnis „hoch“
- Wir befinden uns also im Verzeichnis `/home/` und müssen nur noch den Rest des Pfades anhängen:
- `/home/redha/BuS/sched.txt`

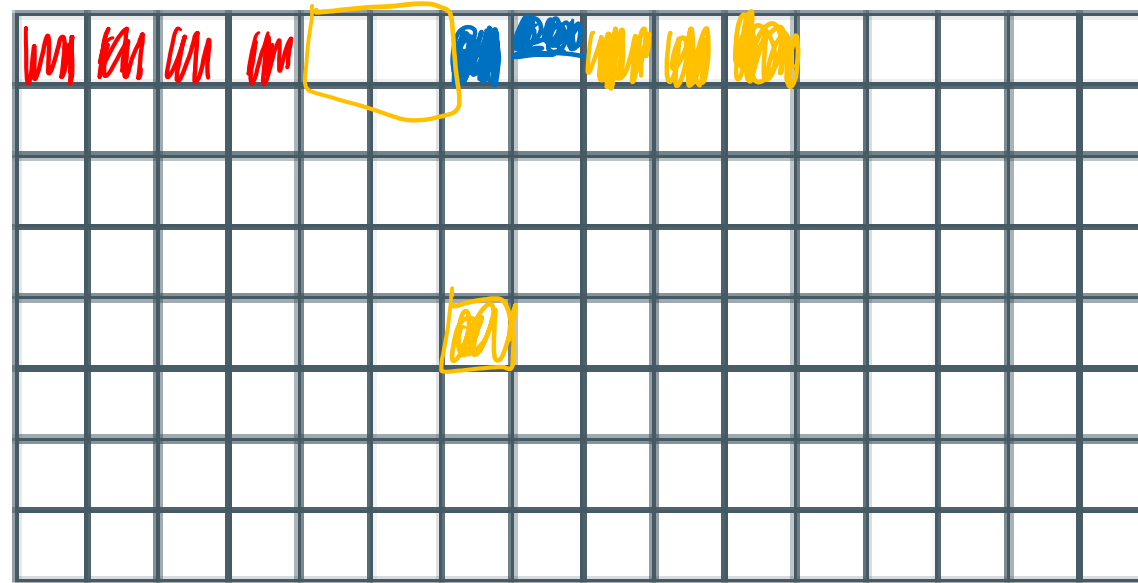


Frage 3:

Contiguous Block Allocation (oder Zusammenhängende Allokation) führt zu Fragmentation auf der Disk.

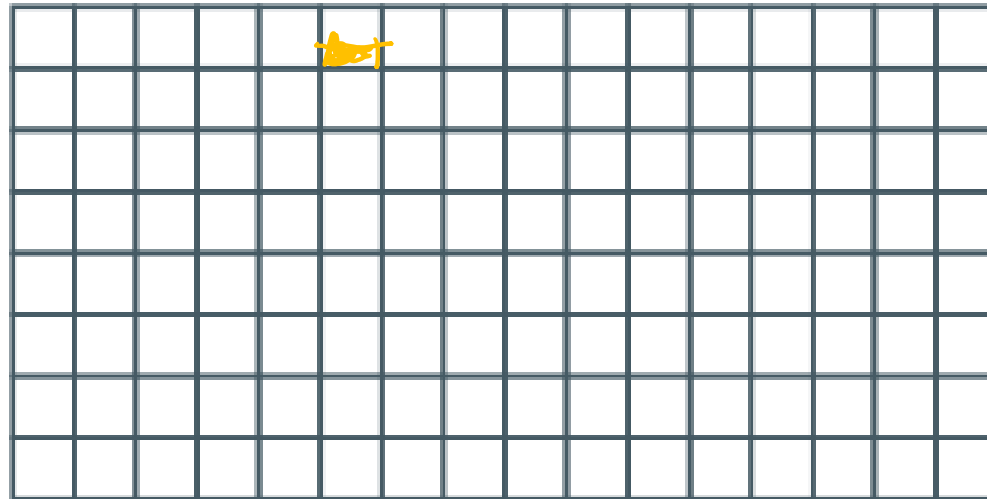
Beschreibt die **interne** und **externe** Fragmentation, die beim Zuweisen und Löschen der Blöcke entsteht.

file 1 - 4
~~file 2 - 2~~
file 3 - 1,5
file 4 - 3



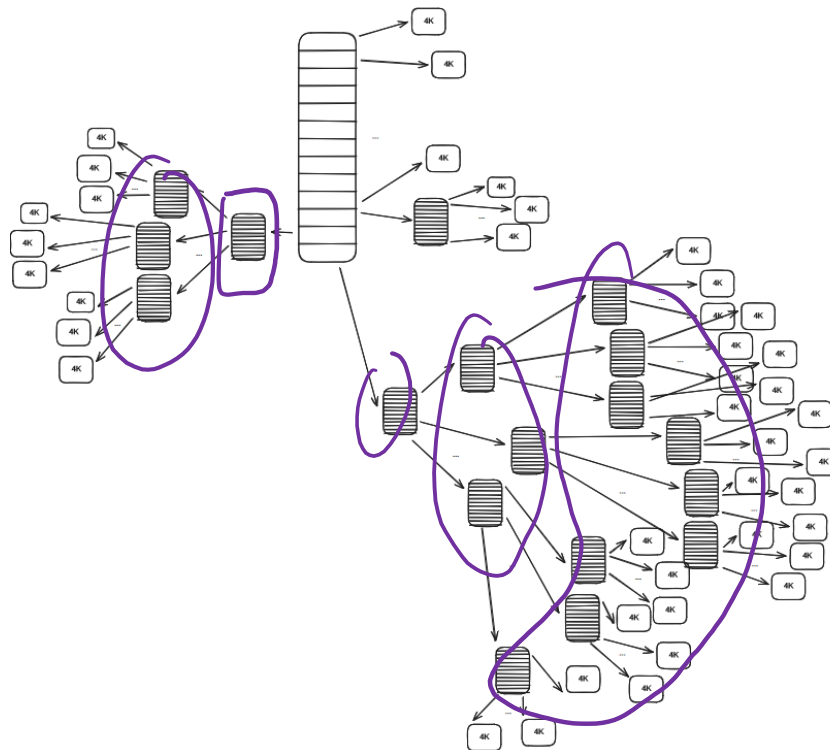
Frage 3: Interne und externe Fragmentation bei Contiguous Allocation

- Interne Fragmentation, weil die Blöcke nicht komplett genutzt werden, da für die Dateien, dessen Größe nicht ein Vielfaches der Blockgröße ist, im letzten Diskblock Speicher verschwendet wird.
- Externe Fragmentation entsteht, sobald wir Dateien löschen und Freiräume zwischen den noch existierenden Blöcken entstehen



Frage 4:

Ein UNIX-Dateisystem hat **4 KiB** Blöcke und **4 Byte** Speicheradressen. Die Inodes haben **10 direkte** Einträge und einen **einfach**, einen **zweifach** und einen **dreifach indirekten** Eintrag. Was wäre hier die **maximale Dateigröße** in Bytes?



4 KiB / 4 Byte
4096 Byte / 4 Byte
= 1024 Pointer pro
Block

Frage 4: Maximale Dateigröße des UNIX-Dateisystems

- 1 Block kann 4 KiB / 4 Byte Speicheradressen umfassen:
 $4 \text{ KiB} = 4096 \text{ Byte} / 4 \text{ Byte} = 1024 \text{ Pointer pro Block}$
- Maximale Größe:
 - $10 * 4 \text{ KiB}$ für die direkten Blöcke
 - $1024 * 4 \text{ KiB}$ für den einfach indirekten Block
 - $1024^2 * 4 \text{ KiB}$ für den zweifach indirekten Block
 - $1024^3 * 4 \text{ KiB}$ für den dreifach indirekten Block
- Gesamtgröße: 4 402 345 713 664 Byte = ca. 4 TB

Frage 5: Directory entries

/bin

/usr/bin

/usr

538819

inode	entry
100282	automake
2189328	base64
2189329	basename
938649	bash
260030	bc
3481110	btrfs
1701644	bzip2
1701645	bzip2recover
2244837	chpasswd
2189336	chroot
938819	sh

inode	entry
3855844	curl
3627359	ip
1698762	kill
2189417	tty
3375120	virsh
3807607	xetex
2242395	xz
2242396	xzcat
938819	sh

inode	entry
626	bin
627	include
628	lib
59563	lib32
2381503	lib64
3855781	libexec
640	local
267721	resources
2381505	sbin
581	share
686	src

Welches sind die Hard Links in diesen Einträgen?

Was würde /bin/sh enthalten, wenn es einen Symbolic Link zur Datei bash wäre?

Frage 5: Hard Links

Directory entries

/bin

inode	entry
100282	automake
2189328	base64
2189329	basename
938649	bash
260030	bc
3481110	btrfs
1701644	bzip2
1701645	bzip2recover
2244837	chpasswd
2189336	chroot
938819	sh

/usr/bin

inode	entry
3855844	curl
3627359	ip
1698762	killall
2189417	tty
3375120	virsh
3807607	xetex
2242395	xz
2242396	xzcat
938819	sh

/usr

inode	entry
626	bin
627	include
628	lib
59563	lib32
2381503	lib64
3855781	libexec
640	local
267721	resources
2381505	sbin
581	share
686	src

- /bin/sh und /usr/bin/sh enthalten dieselbe Inode-Nummer und zeigen somit auf die gleiche Datei → Hard Links

Frage 5: Symbolic Link von `/bin/sh` zu `bash`

Directory entries
`/bin`

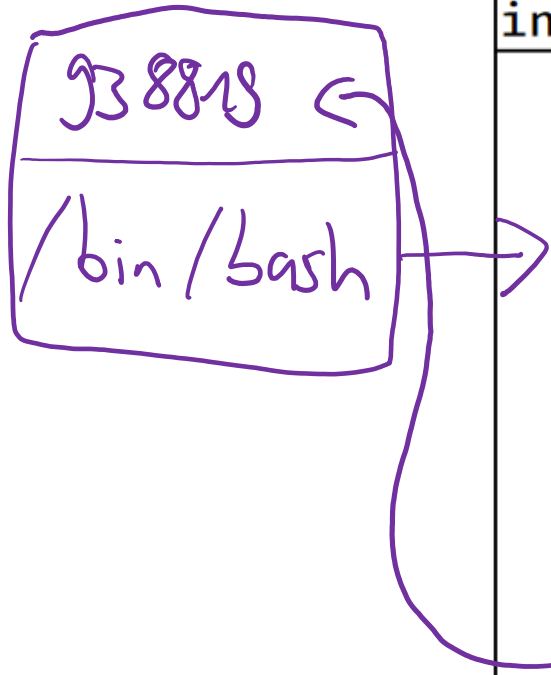
inode	entry
100282	automake
2189328	base64
2189329	basename
938649	bash
260030	bc
3481110	btrfs
1701644	bzip2
1701645	bzip2recover
2244837	chpasswd
2189336	chroot
938819	sh

`/usr/bin`

inode	entry
3855844	curl
3627359	ip
1698762	kill
2189417	tty
3375120	virsh
3807607	xetex
2242395	xz
2242396	xzcat
938819	sh

`/usr`

inode	entry
626	bin
627	include
628	lib
59563	lib32
2381503	lib64
3855781	libexec
640	local
267721	resources
2381505	sbin
581	share
686	src



- `/bin/sh` würde nun einen Pfad zu `/bin/bash` enthalten

Frage 6: Directory entries

/bin

/usr/bin

/usr

inode	entry
100282	automake
2189328	base64
2189329	basename
938649	bash
260030	bc
3481110	btrfs
1701644	bzip2
1701645	bzip2recover
2244837	chpasswd
2189336	chroot
938819	sh

inode	entry
3855844	curl
3627359	ip
1698762	kill
2189417	tty
3375120	virsh
3807607	xetex
2242395	xz
2242396	xzcat
938819	sh

inode	entry
626	bin
627	include
628	lib
59563	lib32
2381503	lib64
3855781	libexec
640	local
267721	resources
2381505	sbin
581	share
686	src

Was würde `/usr/lib64` enthalten, wenn es einen Symbolic Link zum Verzeichnis `/usr/lib` wäre?

Könnte man das auch mit einem Hard Link umsetzen?

Frage 6: Symbolic Link von `/usr/lib64` zu `/usr/lib`

Directory entries

`/bin`

inode	entry
100282	automake
2189328	base64
2189329	basename
938649	bash
260030	bc
3481110	btrfs
1701644	bzip2
1701645	bzip2recover
2244837	chpasswd
2189336	chroot
938819	sh

`/usr/bin`

inode	entry
3855844	curl
3627359	ip
1698762	killall
2189417	tty
3375120	virsh
3807607	xetex
2242395	xz
2242396	xzcat
938819	sh

`/usr`

inode	entry
626	bin
627	include
628	lib
59563	lib32
2381503	lib64
3855781	libexec
640	local
267721	resources
2381505	sbin
581	share
686	src

- `/usr/lib64` wäre eine Datei, nun einen Pfad zu `/usr/lib` enthalten würde
- Hard Links zu Directories sind nicht erlaubt

Frage 7a:

Nennt einen Vorteil von Symbolic Links über Hard Links.

- Symbolic Links erlauben Links ^{auf} zwischen Verzeichnissen und über mehrere Partitionen hinweg, das können Hard Links nicht

Hard links

Symbolic links



Frage 7b:

Nennt einen Vorteil von Hard Links über Symbolic Links.

- Hard Links können nicht „kaputt“ gehen:
Durch den Counter der Hard Links in einer Datei bleibt die Datei nach Löschung einer der Links mit Counter > 0 noch erhalten und der Link funktioniert weiter
- Bei Symbolic Links ist das nicht so:
Wenn man die Originaldatei löscht, zeigen die Symbolic Links, die auf diese Datei gezeigt haben, ins Nichts

Frage 8:

Wie viele Disk-Operationen sind notwendig, wenn wir die Inode für die Datei mit dem Pfad /usr/ast/courses/os/handout.t laden wollen?

Nehmt an, dass die Inode für unser Root-Directory schon im Hauptspeicher liegt. Der Rest des Pfades muss erst vollständig geladen werden. Nehmt außerdem an, dass alle Verzeichnisse in einen Disk-Block passen.

Wie könnten wir die Anzahl der Disk-Operationen reduzieren?

Frage 8: Anzahl der Disk-Operationen beim Aufruf von `/usr/ast/courses/os/handout.t`

- 1 fürs Laden vom Block Directory von `/` (dort wo die Inode-Nr von `usr` steht)
- 1 fürs Laden der Inode von `/usr`
- 1 fürs Laden vom Block Directory von `/usr` (Inode-Nr von `ast`)
- 1 fürs Laden der Inode von `/usr/ast`
- 1 fürs Laden vom Block Directory von `/usr/ast`
- 1 fürs Laden der Inode von `/usr/ast/courses`
- 1 fürs Laden vom Block Directory von `/usr/ast/courses`
- 1 fürs Laden der Inode von `/usr/ast/courses/os`
- 1 fürs Laden vom Block Directory von `/usr/ast/courses/os`
- 1 fürs Laden der Inode von `/usr/ast/courses/os/handout.t`
- Insgesamt 10 Disk-Zugriffe
- Zum Reduzieren der Disk-Zugriffe haben moderne Systeme oft einen Cache zwischen den Directory-Einträgen und der Inode-Nr

Manual Pages

Schaut euch den syscall `open` im Manual an.

Was sind die übergebenen Argumente, was ist der Rückgabewert?

Tipp

Im Linux Terminal könnt ihr, um die Manual pages aufzurufen, den Befehl `man <Sektionsnummer> <command_name>` nutzen.

In unserem Fall benötigen wir für unsere gesuchten Informationen die 2. Sektion vom Manual von `open`. Wir schreiben also `man 2 open`.

Falls ihr keinen Zugriff auf ein Linux Terminal habt, gibt es alle man pages auch im Internet.

Man Pages: `open`: Argumente, Rückgabe und Funktionsweise

- Argumente:
 - `String` für den Pfad der Datei
 - `int` für die Flags
 - optional: `mode` für das Erstellen einer Datei
- Rückgabewert:
 - Erfolg: File Deskriptor der zu öffnenden Datei
 - Error: -1

Manual Pages

Wie benutzt man diese Funktion, um eine neue Datei zu erstellen?

Was macht das Flag `O_TRUNCATE` (`O_TRUNC`) ?

Wie kann man mehrere Flags kombinieren?

Man Pages: `open`: Argumente, Rückgabe und Funktionsweise

- Datei erstellen mit `O_CREAT`
 - bei Nutzung von `O_CREAT` brauchen wir einen Mode, wir nehmen hier `S_IRWXU` (User hat read/write/execute Permissions)
 - `int fd = open("new_file", O_CREAT, 00700);`
- Flag `O_TRUNC` kürzt die zu öffnende Datei auf Länge 0 (wenn die entsprechenden Berechtigungen erteilt sind)
- Mehrere Flags können mit `|` verknüpft werden
 - `int fd = open("file", O_RDWR | O_TRUNC);`
könnte in die Datei schreiben und die gesamte Datei leeren

File Deskriptoren, File Tables und Inodes

Donnerstag, 30. Mai 2024 10:29

Please note that `file1.txt` and `file2.txt` are hard links. Suppose that no file descriptor is in use at the start of the program, except for the standard input, standard output and standard error.

Task

At the specified lines (14 and 24), draw the state of:

- The file descriptor table for each process.
- The associated file structures (including inode, mode, and cursor).
- The corresponding inode.

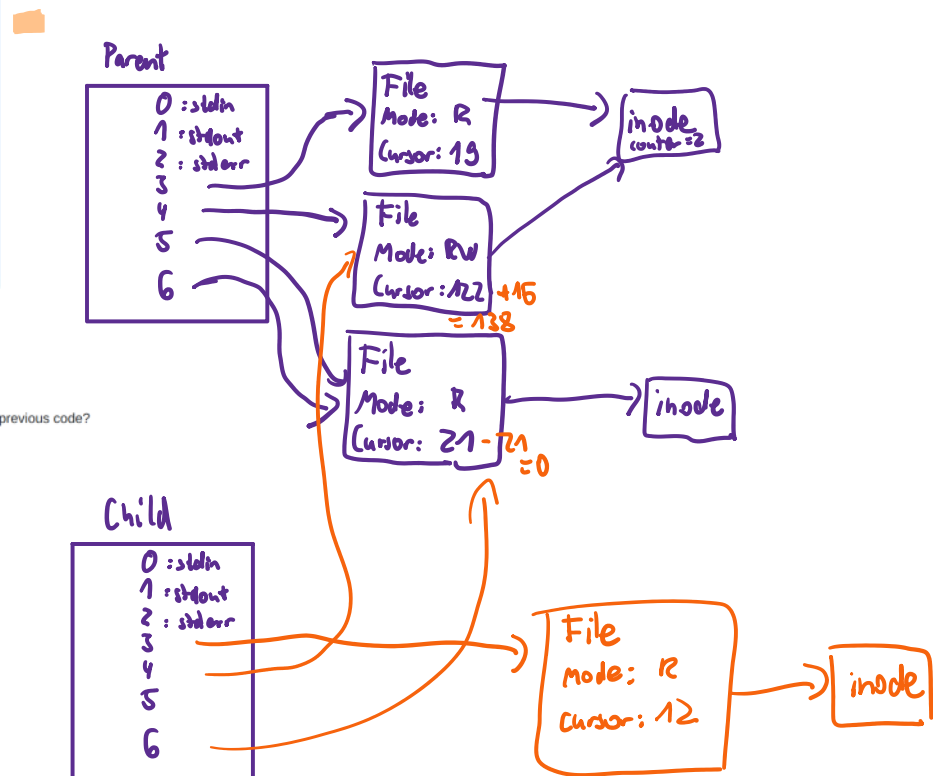
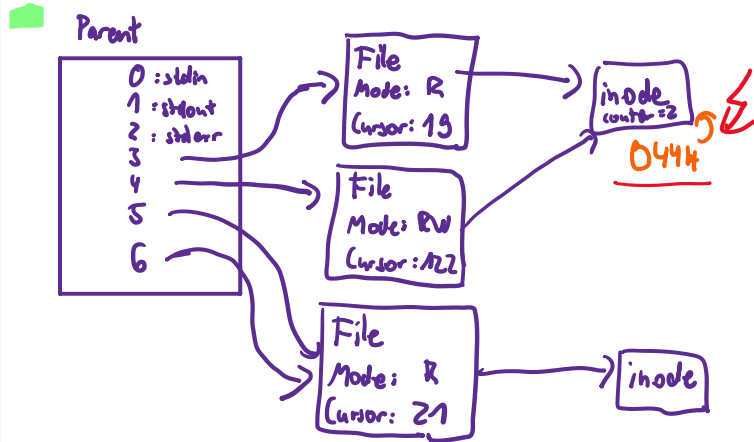
Example: UNIX file descriptors

When opening a file, UNIX systems return an integer called *file descriptor*.

The actual file descriptor is stored in the kernel, in the process' PCB, in a table. The integer returned to user space is the index in the table.

Descriptors 0, 1 and 2 have a special meaning: standard input (*stdin*), output (*stdout*) and error (*stderr*) respectively.

```
1 int main()
2 {
3     char buffer[16];
4
5     {int fd0 = open("file1.txt", O_RDONLY);
6       int fd1 = open("file2.txt", O_RDWR);
7       int fd2 = open("file3.txt", O_RDONLY | O_TRUNC);
8       int fd3 = dup(fd2);
9
10      lseek(fd0, 19, SEEK_SET);
11      lseek(fd1, 122, SEEK_SET);
12      lseek(fd2, 5, SEEK_SET);
13      read(fd3, buffer, 16);
14
15      if (fork() == 0) {
16          close(fd0);
17          close(fd2);
18          int fd4 = open("file5.txt", O_RDONLY);
19
20          lseek(fd4, 12, SEEK_SET);
21          write(fd1, BUFFER, 16);
22          lseek(fd3, -21, SEEK_CUR);
23
24          close(fd1);
25          close(fd3);
26          close(fd4);
27          return;
28      }
29
30      close(fd0);
31      close(fd1);
32      close(fd2);
33      close(fd3);
34
35 }
```



Questions

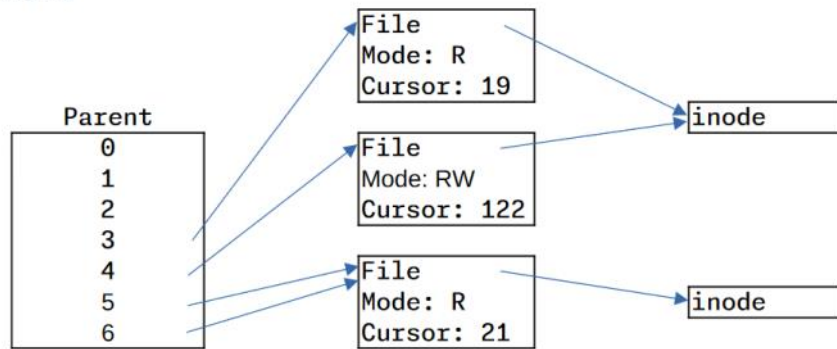
1. If the file permission of `file1.txt` was `0444`, what would be the result of executing the previous code?

Examples of POSIX permissions		
Text	Octal	Description
-----	0000	Regular file, no permissions
-r--r--r--	0664	Regular file, owner and group can read-write, others are read-only
-rwxr-xr-x	0755	Regular file, everyone can read and execute, owner can modify
-r--r--r--	0644	Regular file, owner can read-write, everyone else is read-only
drwxrwxr-x	0775	Directory, owner and group can do everything, others can only list entries and check content
drwxr-xr--	0754	Directory, owner can do everything, group can list and check, others can only list entries
drwxrwx--x	0771	Directory, owner and group can do everything, others can only traverse, but not list entries

r--r--r-- Only read für alle

Musterlösung:

Diagrams at line 14



Diagrams at line 24

